Prepared by Sumaiya Tarannum NOOR Date: 12 11 2026

Lecture 5 – CUDA Programming
Suggestion Questions & Answers

1. What is the SIMT execution model in CUDA, and how does it differ from SIMD?

Ans.

SIMT (Single Instruction, Multiple Threads):
Programmers write scalar instructions for independent threads; the GPU groups these threads into 32-thread warps.
Each thread has it's own registers, data and control-flow state.

SIMD (Single Instruction, Multiple Data):
One vector instruction operates on multiple data elements simultaneously.

fundamental difference:

SIMD: Software expresses vector operations; hardware executes them across vector lanes.

SIMT: Software-express individual thread operations; hardware groups threads into warps for execution.

Both exploit data parallelism, but SIMT hides much of the hardware

execution width behind the thread abstraction.

2. What is a warp in CUDA, and how are threads organized into warps?

Ans. Warp is a 32 threads thread group.
So, a warp is a collection of 32 threads.

CUDA uses SIMT, where programmers write code for individual threads, but hardware schedules and executes them in groups called warps.

Each warp contains 32 threads, managed and scheduled by a Streaming Multiprocessor (SM).

Threads within a warp initially execute same kernel instruction but operate on their own data.

A warp achieves maximum utilization when all 32 threads follow the same execution path.

3. How does GPU hardware multithreading help hide memory and instruction latency?

Ans. when a thread block is assigned to an SM, it is partitioned into warps of 32 threads.

Each warp is scheduled for execution by an SM warp scheduler.

When one warp stalls, the scheduler can execute instructions from other ready warps.

Higher occupancy which is the measured proportion of resident warps by maximum supported warps, supports latency hiding.

Occupancy = Resident warp / maximum supported warps.

It provides a larger pool of work that is available for execution.

~~This is Independent instru~~

~~Higher occupancy may require~~

~~fewer~~

Higher occupancy means the resident threads may get enough registers to work with. which ~~means~~ means independent instruction execution from ready warps become easier.

This is how GPU hardware multithreading help hide memory and instruction latency.

4. What is the main difference between SIMT programming and CUDA tile programming?

Ans. The main difference between SIMT and Tile programming is shown below

SIMT	Tile Programming
Programmers write ~~code~~ instructions for ~~individual~~ individual threads and explicitly maps threads to data using threadIdx, blockIdx and blockDim.	Programmers describe operations on multidimensional data ~~as~~ tile and compiler ~~executes~~ ~~them~~ distributes the work among threads in a block.

5. How do arrays and tiles differ in terms of memory storage and mutability?

Ans.

Arrays – Data Stored in GPU Memory

→ Arrays are multidimensional collections stored in device (global) memory
→ Each array has a defined shape and data type.
→ Arrays are ~~read~~ mutable: Threads can modify their elements through store operations.
→ Arrays provide persistant data storage across tile operations.

Tiles – Temporary Data Used for Computation

→ A Tile is multidimensional collection of values local to a single thread block.
→ Tiles are immutable: Operations produce new tiles instead of modifying existing ones.

Sa Su Mo Tu We Th Fr Date:

→ The compiler determines where tile values reside, potentially using registers or shared memory.

→ Tile dimensions must be powers of two and known at compile time.

→ Tiles are created and used inside the kernel; they cannot be passed directly as kernel parameters.

6. What is the purpose of --global-- keyword in CUDA C++?

Ans. The code for a kernel is specified using the --global-- declaration specifier. This indicates to the compiler that this function will be compiled for the CPU in a way that allows it to be invoked from a kernel launch. This invoke operation happens in host which is CPU.

A kernel launch is an operation which starts a kernel running, usually from the CPU.
Kernels are functions with void return type.

7. Explain the roles of threadIdx, blockIdx, and blockDim in CUDA. How is a thread's global index calculated.

Ans.

threadIdx : threadIdx gives the index of a thread within its thread block. Each thread ~~block~~ in a thread block has a different index.

blockIdx: blockIdx gives the index of a thread block within the grid. Each thread block will have a different index.

blockDim: blockDim gives the dimensions of the threadblocks, which was specified in the execution configuration of the kernel launch.

A threads global Index is calculated using threadIdx, blockIdx and blockDim

global index = blockIdx.x * blockDim + threadIdx.x

Date:

8. what is Unified Memory in CUDA, and what is the purpose of cudaMallocManaged()?

Ans.

Unified Memory in CUDA is a single memory space that is accessible by all GPUs and CPUs in a system.

To allocate data in unified memory, we call cudaMallocManaged(), which returns a pointer that we can access from the host (CPU) code or device (GPU) code. To free the data, ~~just pass~~ we need to just pass the pointer to cudaFree().

—— x ——

—— o ——